

TEAM PRESENTATION

A simple guide for the final-day presentation

No prior project knowledge needed. This guide gives the team a clear story to tell, explains the data structures without jargon, and walks through a short live demo.

The project in one sentence

FoodShare is a website where people can list surplus food, NGOs can request it, and a coordinator can organize pickup and delivery — while real C data structures manage the important operations.

What you will find inside

- A short opening script to explain the problem and your solution.
- A simple example that walks through one donation from start to delivery.
- Plain-language explanations for the stack, queue, deque, heap, tree, linked list, and graph.
- A safe, step-by-step demo plan and short answers to likely questions.

Live site

<https://smart-food-donate.vercel.app>

The /c-demo page is the clearest place to show the C structures. Render hosts the API behind the scenes; the audience can stay on the FoodShare site.

1. The story to tell

Start with the real-world problem, then explain what your solution does

The problem

Restaurants, hotels, cafes, and community kitchens may have safe surplus food at the end of the day. Nearby community groups may need meals. Without a simple way to share what is available, how much there is, and when it expires, food can be wasted while people go hungry.

Our idea

FoodShare gives donors a place to list surplus food and its expiry time. NGOs can browse the offers and request servings. The app then keeps track of request order, pickup priority, and delivery progress.

A 30-second opening you can say

Opening script

“Our project is FoodShare, a food donation and distribution demo. It connects surplus food donors with NGOs. A donor posts food with a quantity and expiry, an NGO requests it, and the team can prioritize pickup and track delivery. What makes our project educational is that the C data structures are not just diagrams: they actually handle ordering, searching, dispatch, history, routes, and network traversal.”

Who uses the demo?

Person	What they do
Donor	Lists available food, servings, pickup place, and expiry.
NGO	Finds available food and requests servings.
Coordinator	Reorders the dispatch lane and approves the next pickup.
Volunteer / learner	Moves delivery progress forward and inspects the structures.

Keep the promise accurate

Say this is a learning prototype and workflow demonstration. Do not describe it as a verified food-safety system, a real road-navigation map, or a secure production service.

2. One donation, from offer to delivery

Use one concrete example so the audience can follow every step

Imagine Sunrise Hotel has 24 breakfast packs left. They should be picked up within 45 minutes. A nearby relief NGO wants five servings.

Moment	What the user does	What the app/C code does
1. Offer	Enter “Breakfast packs”, Sunrise Hotel, 24 servings, pickup place, expiry.	Validate the offer; assign an ID; index its ID in the balanced tree; add its expiry to the min-heap.
2. Request	NGO selects the food and asks for five servings.	Check role, availability, ID, and quantity; mark the offer claimed; put the request at the FIFO rear.
3. Dispatch	Coordinator looks at requests and can move one to the front or back.	Build a separate deque so urgent or manually prioritized work is handled next.
4. Deliver	A team member advances the route step by step.	Update request/food status, record activity, and extend the linked route.
5. Explore	Learner searches, inspects, or runs BFS/DFS.	C returns the actual tree search, data-structure order, graph edges, and traversal result.

The status words, simply

Status	Meaning
AVAILABLE	Food is listed and can be requested.
CLAIMED	An NGO has requested it.
COLLECTED	The pickup stage has been reached.
DISTRIBUTED	Delivery is complete.

Say this while presenting the example

“The important thing is that each new action updates more than the screen. The server changes its C data structures too. We can inspect those structures and see why the next food or pickup is where it is.”

3. The C structures, without jargon

Learn one everyday analogy and one reason each structure fits

Structure	Everyday picture	Why FoodShare uses it
Stack	A pile of plates: take the top one first.	Recent actions: the newest action is shown first (LIFO).
Queue	A line at a counter: first person in line is served first.	Pending requests keep their original arrival order (FIFO).
Deque	A line with doors at both ends.	Dispatch can add or remove work at the front or back; urgent requests can move forward.
Priority queue / min-heap	A tray that keeps the soonest deadline ready at the top.	Food closest to expiry is surfaced first.
Balanced search tree	A sorted filing tree: compare an ID, then go left or right.	Find a donation by numeric ID and show how deep the search went.
Linked list	A chain where each link points to the next stop.	Represent ordered delivery milestones from the first step to the current one.
Graph	A connection map made from dots and lines.	Show links between donors, the food hub, and NGOs; compare BFS and DFS.

Three distinctions worth emphasizing

- Queue versus deque: the queue answers “who asked first?” The deque answers “who should dispatch next?”
- Tree versus heap: the tree is for finding a particular donation ID. The heap is for finding the most urgent expiry.
- BFS versus DFS: BFS explores nearby connections first; DFS follows one path deeper before backtracking.

A sentence to repeat

“We chose each structure because it matches a different job; one structure is not being used to pretend to do everything.”

4. A no-stress live demo

A short sequence you can rehearse before the presentation

Before the presentation

- Open <https://smart-food-donate.vercel.app/c-demo> on the presentation device and leave it open.
- If the page says it is connecting, give the free API time to wake; the first request after idling may be slow. Stay on the FoodShare page.
- Click “Load complete example” once the API is connected. It fills the view with sample food, pickup requests, and delivery progress.
- Keep this guide and the public project report available as backup. Never type personal information into the demo.

Live sequence (about 3 minutes)

Time	Do this	Say this / point out
0:00	Show the page title and C API badge.	“This is the public FoodShare structure demo. The page is connected to our C backend.”
0:20	Click Load complete example.	“This gives us data to demonstrate without manually filling every structure.”
0:45	Point at the priority queue.	“The soonest-expiring food appears first because the C min-heap prioritizes expiry.”
1:10	Compare Queue and Deque cards.	“The queue retains request arrival order; the deque gives us a separate urgent-first dispatch order.”
1:40	Show the BST, then search for “Breakfast packs”.	“The page finds the matching donation, then the C tree searches by its numeric ID and reports depth.”
2:05	Select a graph node and compare BFS + DFS.	“The same network can be explored breadth-first or depth-first; watch the visit order change.”
2:35	Point to the linked route and stack.	“The route is a chain of milestones. The stack keeps the newest action on top.”

If the C API is sleeping

A long first load is expected on the free Render service after inactivity. Do not keep clicking controls repeatedly. Wait for “C API connected” or refresh once the page has had time to reconnect. If it cannot wake during the presentation, explain the architecture with the diagrams and use the sample steps in this guide.

5. How to prove it is really C

A direct answer to the most important technical question

The short answer

“Is C really doing the work?”

“Yes. The UI is built in React and TypeScript, but the shared donation and request operations call a native C HTTP server. That server uses the implementations in c-backend/ds.c. The project also compiles the same C structure library to WebAssembly for browser-based learning features.”

What is where

Part	Plain explanation	Project location
React + TypeScript	Forms, pages, diagrams, and button interactions.	src/
C data structures	The arrays, links, tree operations, heap ordering, and graph traversal.	c-backend/ds.c and ds.h
C HTTP server	Receives browser requests, checks inputs, calls C structures, returns JSON.	c-backend/server.c
WebAssembly build	Runs the same C algorithms inside a browser for lab/product traces.	c-backend/wasm_api.c and public/ds.wasm
Native tests	Exercises behavior and boundary cases for the structures.	c-backend/tests.c

A helpful way to describe the architecture

The browser is the control panel: it collects information and draws the result. The C server is the operator: it validates the workflow and calls the structures. The structures are the working tools: they decide order, look up IDs, record history, and traverse connections. The browser then displays the answer returned by C.

A little evidence to mention

- The health endpoint reports engine: c.
- Donation creation inserts an ID into the C tree and expiry into the C heap.
- Pending pickup requests are returned in C FIFO order; dispatch comes from a C deque.
- The C test command reports 385 checks across six core structures.

6. Questions you may be asked

Short answers for the final discussion or viva

Question	Clear answer
Why use several structures instead of one?	Each structure fits a different job: order, urgency, lookup, route links, activity, or relationships.
What is the difference between the queue and deque?	The queue preserves first-asked-first order. The deque lets dispatch work be prioritized from either end.
Why a heap for expiry?	A min-heap keeps the smallest expiry time at the top, so the most urgent item is easy to inspect.
Why use a tree if donation IDs are numbers?	The tree demonstrates indexed searching; it can find an ID by comparing keys and moving left/right. It also reports search depth.
Why balance the tree?	IDs arrive in increasing order. Without rotations, a basic BST can become a long chain. The AVL-style balancing keeps it shallow.
What do BFS and DFS show?	They visit connected graph nodes in different orders: nearby-first versus one-path-deep-first.
Is it using a real map or road directions?	No. The graph is a small relationship network. The delivery picture is illustrative, not live navigation.
Where is the data stored?	In memory inside the C server. It is shared while that instance runs, but restarts can clear it.
Is the sign-in secure?	No. These are fixed demo roles stored in the browser, not production authentication.
What happens if the backend has been idle?	The free Render API can sleep. Its first request may take about a minute to wake, while the visitor remains on the FoodShare site.
What would you improve next?	Add proper server-side accounts/authorization, persistent database storage, stronger production deployment controls, and real operational routing if needed.

7. Team plan and final checklist

Divide speaking if useful; make sure everyone can explain the whole story

A flexible speaking split

Adapt this to the number of presenters. These are suggested topics, not assigned people.

Part	Suggested speaker topic	Key point to land
Opening	Problem, target users, and the FoodShare idea.	Surplus food and community need should connect faster.
Technical idea	Explain the C structures with the simple analogies.	Each structure has one job in the real workflow.
Demo	Run the prepared steps on /c-demo.	The C API response powers what the page shows.
Closing / Q&A	Limits, testing, future work, and audience questions.	It is a C-learning demo, not a production food service.

Everyone should be ready to answer

- What the project is and who uses it.
- How one donation moves from AVAILABLE to DISTRIBUTED.
- At least one difference between queue, deque, heap, and tree.
- Why you can say the backend genuinely uses C.
- What will happen if the free backend sleeps or restarts.

Final-day checklist

Check	Done?
Open the Vercel site and verify the page loads.	&
Confirm C API connected; allow time if the free backend is waking.	&
Load sample data and verify the main panels have examples.	&
Practice the 30-second opening and the 3-minute demo once.	&
Make sure each teammate can explain their section in their own words.	&
Avoid personal data; be honest about in-memory storage and demo security.	&

The closing line

“FoodShare shows how familiar data structures can support a practical workflow: matching surplus food with requests, prioritizing what is urgent, and making the C operations visible enough to learn from.”